Chapter 1 — Introduction

- What evidence, beyond correlation, supports 'more data, better model'? Couldn't compute and architecture explain the gains instead?
- Why differential privacy specifically, vs. SMC, homomorphic encryption, anonymization, or FL alone? How do DP and FL relate in the framing?
- What is the single methodological thread unifying contact tracing, hidden-state DP-SGD, MoE LLMs, and DP in-context learning?

Chapter 2 — NTTW (communication-efficient tracing)

-What does FN actually optimize on loopy graphs? Any convergence or correctness guarantee, and when does its fixed point differ from the true posterior?

- No exactness guarantee like BP for trees
- Fixed points always exist, though: the update is a continuous map from a product of simplices to itself, so Brouwer's theorem applies.
- Intuitively, each step is an average, so will likely converge

-All evidence is simulator-based. How would results transfer to a real pandemic: misspecified dynamics, noisy contact graphs, partial adoption?

-Is dropping backward messages principled or an empirical fix? Does it bias the posterior, and when would backward messages matter?

Why is FN a good inference method for the model, and what does it converge to?
Rosen-Zvi et al. derived FN for an undirected Ising model.

- What objective FN optimizes, if any.
- Whether it has a fixed point or convergence guarantees on loopy directed graphs.
- Why five rounds is enough.

One or two rounds already beat random, why five. RQ1 asks for the *minimal* bandwidth, so also: "Is there a lower bound, or only an empirical operating point?"

Dropping bwd messages: is that a general property or an artefact of the setting?
You justify it by noting that 99.9% of backward messages are uniform. Expect follow-ups:

- Was that before or after 4-bit quantization? Rounding could collapse small deviations
- Under which regimes would backward messages matter, such as dense or accurate testing or small households?
- Does dropping them break source attribution? That is a core part of contact tracing.

Thesis outlines three dilution factors (many contacts, noisy tests, scarce tests).

Is the comparison with Gibbs and BP fair, and is bit-counting the right measure?
Several angles are likely here:

- **Gibbs setup.** Gibbs gets only 10 burn-in steps, and each sample is counted as 2 bits. The mixing plot shows a slope of −1/3. Standard Monte Carlo error decays as N^(−1/2), so explain why the slope is slower.
  Gibbs uses PrivacyForFree, like exponential-mechanism on the sample for each user, while BP and FN are about the messages between users. That's more comparable to the DNA method then
- **The OpenABM gap.** On OpenABM, FN reaches 0.06% PIR against 16.9% for BP. That two-orders-of-magnitude difference looks like a threshold effect: the outbreak is either contained or it isn't. The wide interval at 1% FPR, [0.8, 21.3], supports that reading. is bimodal and how robust it is across seeds.
- **Why count bits at all.** In a real deployment, encryption and protocol overhead are hundreds of bytes per message. Is the number of *rounds* the real cost, or bits?

*Simulator validity. The headline results (2–100x peak-infection reduction) rest entirely on CRISP, OpenABM, and Covasim — the latter two tuned to UK/US demographics. Beyond parameter fitting, what validation did you do, and how do you know the gains aren't an artifact of simulator misspecification?

*Behavioral realism. You assume 100% test follow-up and full adoption, yet real adoption was single-digit and notified users change behavior (risk compensation, self-quarantining). Chapter 4 mentions "agents not following the protocol" — what happens to the privacy-utility trade-off under partial adoption and behavioral feedback?

*Fairness of the DP noise. DP can hit minorities hardest. Since DP noise distorts who gets tested, did you measure disparate impact across the age/household/occupational strata in OpenABM/Covasim: does DPFN widen or narrow health inequities?

*In Chapter 2 the inference model and data generator are essentially identical, and in Chapter 4 train and test on the same simulator. How do you know you're not learning simulator artefacts? What would validation against real-world data look like?

*Factorised Neighbours assumes neighbours' beliefs are independent. Households and workplaces are highly clustered, so neighbours are strongly correlated. When does this break down?

*Bluetooth proximity is very noisy: through walls, in crowds, depending on phone position. How sensitive are FN and DPFN to false-positive and false-negative contacts? In a dense venue with 100+ contacts, what happens?

Simulation related:

- How do you know that DNA learns useful epidemiological patterns rather than peculiarities of OpenABM?
- What evidence would convince you that the improvements in the simulators would also occur in a real epidemic?
- Which properties of the contact network would make Factorized Neighbors fail?

- You motivate differential privacy through increased adoption. How would you model and test the link between privacy guarantees, adoption, and epidemic control?
- Why should testing the people most likely to be infected be the best epidemic-control policy?
- Could the system reduce population-wide infections while making outcomes worse for particular social groups?

Ablations:

- Chapter NTTW
    - Noisy testing, different FPR and FNR
    - Stale phone scenario (not giving updates half the time)
    - Number of updates
    - Time window of inference
- PYS
    - Noisy testing, different FPR, FNR
    - Different scale 50k, 100k, 500k, 1M
        - 50k and 100k start with 25
        - 500k and 1M start with 50
    - Loss to follow-up (i.e., user tests positive but does not isolate)
    - Number of tests available
    - Time window of inference
- DNA
    - Noisy testing
    - Follow protocol

Social impact:

- The predictions change people's behaviour, which changes what you are predicting. What happens to the system under that feedback?
- What happens when users strategically manipulate the system rather than simply fail to follow its recommendations?
- Installing the app, reporting a test and quarantining when flagged are costly for me and benefit others. That's a classic social dilemma. The thesis assumes privacy is the barrier to adoption. But even with perfect privacy, why wouldn't a rational agent free-ride? What mechanism keeps people cooperating?
- Who should bear the costs of privacy-preserving epidemic control, and what would a fair allocation look like?
- Could protecting privacy weaken the reputation and accountability mechanisms that sustain cooperation?
- In the multi-department setting, some departments contribute much more knowledge to the shared component than others. Why would a data-rich department join? Does every department gain from transfer, or do some subsidise others? Is the benefit distributed fairly, for example by a Shapley-style measure, and could a department free-ride by contributing little?

Chapter 3 — PYS (DP contact tracing)

- How does ε=1 per message compose across repeated contacts, days, and the 14-day window? What is a user's true worst-case privacy loss?
- You assume unmodified inference in a TEE — what does DP add on top of a trusted execution environment?
- At 25% FPR, DPFN's PIR jumps from 0.6‰ to 46.6‰. Doesn't that undermine deployment, and is 'just run more tests' a realistic policy answer?

What is the actual privacy guarantee per person over a pandemic, not per message?
This is the most likely hard question for this chapter. Points they may raise:

- **Composition over time**
- **Repeated contacts**

Prepare for a Q about privacy-unit choices – mentioned in our own conclusion section

PYS threat model relies on a trusted execution environment. If the TEE holds, why do you need DP? And if it doesn't, what does DP give you?

How were hyperparameters chosen, and are the comparisons equal?

- **Tuning on the test environment**
- **Unequal test budgets.** DPFN+ uses 15% daily tests while the no-privacy column uses 10%, so the comparison is not like-for-like.
- **Per-message baseline.** With γ=0.01, the logit-domain sensitivity is about 9.2. Is that baseline a strawman?
- **Scaling result.** DPFN's PIR falls from 2.5‰ to 0.2‰ as population grows, while traditional tracing stays flat. The number of initially infected people also changes at the larger scales. Expect "why?"

Chapter 4 — DNA (neural augmentation)

- What patterns does the network learn beyond SEIR, and would a module trained on OpenABM transfer to a real virus with different dynamics?
- How tight is the sensitivity bound $\gamma_u \cdot p_1 \cdot (1+1/C_t)$? How do you handle users with very few contacts, and is the Lipschitz projection truly negligible?
- You train on AUC but deploy on peak infection rate — why not train end-to-end on the control objective (RL)? And explain the corrected ε>1 footnote.

Does the sensitivity proof account for normalization by the observations?
The proof treats F as a sum of products that is affine in $(1-p_i\mu_i)$. But F is a posterior conditioned on the user's own test results. But yes I verified they sum to 1

- Theorem 1 states the sensitivity *equals* $p_i\gamma_u$, but the proof only shows ≤.
- It's of course if you talk about the max or sensitivity

Where does the training data come from in a real pandemic?
The network is trained and evaluated on OpenABM with different seeds, so the sim-to-real gap is untested. Expect these follow-ups:

- If you trained on real data, you would need DP-SGD, or DP reinforcement learning and the privacy accounting would change.
- The training data was generated under one testing policy, but deployed predictions change who gets tested. How do you handle that closed-loop distribution shift?

What is the statistical model actually contributing, and why this architecture?

- **The AUC numbers.** FN alone gets 77.0 AUC, the Lipschitz network alone 80.3, and the combination 83.1. So why keep FN, given it carries the larger sensitivity term?
- **Choice of activation.** A spectrally normalized ReLU network is known to lose expressivity. The cited Anil et al. paper uses GroupSort for this reason. Why ReLU?
- **Worst-case contact count.** In the own attack scenario, the victim is the adversary's only contact, so C□=1 and the neural term doubles the noise. Is C□ treated as public? It is data-dependent, and the noise scale depends on it.
- **The ε correction.** Expect a question about the footnote on recalculated ε values: what changed, and does it affect any conclusions?

What features did we add in the DNA chapter? Was it score only or also age?
Score only, but subsequent MSc students showed a bunch better results with other features

Cross-cutting

- Is ε=1 per message 'private' once summed over all a user's status influences?
- DP can disproportionately hurt minority groups — how do you respond to the bias-amplification critique in a healthcare setting?
- You suggest DP should inform GDPR's right to erasure — how would anyone verify a model was really trained with DP?

Cross cutting on the context tracing chapters

A few tips on where the pressure points are:

- **Introduction:** a "big picture" question (why DP over other privacy techniques) and a "coherence" question (what unifies the thesis).
- **NTTW:** the FN algorithm is the novel contribution, so the theory question — *what does FN optimize on loopy graphs, and when is its fixed point wrong* — is likely. Also the simulator-only evaluation and the backward-message ablation

- **PYS:** composition. The text itself discusses ε grows K-fold under repeated contacts. The TEE assumption (DP on top of a trusted component) is the second likely Q.
- **DNA:** the gap between training objective (AUC) and deployment metric (PIR) is a classic Q, and the footnote about the recalculated ε values is another.
- You arge for decentralization, but then try to find the TOP-RANKING-COVIDSCORES which is a global op? -> we cite two Secure Multiparty Computation approaches in the Appendix of PYS. A threshold-based mechanism was investigated by two MSc students. Still need a DP way to find the Current Infection Rate, but that could probably be done with RAPPOR.

Cross-cutting questions (likely from at least one examiner)

- **Full adoption.** Every experiment assumes full app adoption and a known contact graph, yet the motivation of the thesis is low adoption. How does performance degrade at 20–40% adoption?
- **Inconsistent setups.** Population size (10k, 10k, then 100k), test fraction (15%, 10%, 8%) and p_1 all vary between chapters.
- **No real-world validation.** Everything is simulated, and both simulators are calibrated to the UK or US.

Textual issues worth fixing or preparing for

These are small, but examiners love them:

- **ASCII claim in Chapter 2.** The introduction says five 4-bit messages per day cost less than one ASCII character per contact. That is 20 bits against 7–8 bits.
- **Bit counts.**
- **Quantization.** The formula rounds values, which makes it mid-tread, not mid-rise. On [0,1] it also produces $\kappa+1$ levels, not κ, which would need $c+1$ bits.
- **Swapped test fractions.** Chapter 3's main text says OpenABM tests 10% and Covasim 2% daily.
- **p_1 values.** The PYS appendix gives $p_1 \approx 0.01$ from OpenABM, while Chapter 4 says "typically around 0.02". Higher p1 to more effective contact tracing

Chapter Cvx ReLU

gap between the theory and experiments deliver.

*You dropped the cone constraints, replaced group-ℓ2–ℓ1 with squared ℓ2, and sampled P≪M patterns — is this still a convex reformulation of the ReLU network, or just a random-feature GLM? What is the approximation gap as a function of P?

*NoisyCGD's ε protects only the final model; DP-SGD's ε protects the whole trajectory — is comparing at matched ε really apples-to-apples? And when does the hidden-state threat model actually hold in practice?

*λ plays a double role: larger λ speeds 'forgetting' (tighter privacy bound) but hurts accuracy. How did you set λ, does the advantage survive λ tuned purely for accuracy, and is the privacy cost of hyperparameter tuning accounted for?

- The λ tension. $\eta\lambda = 2 \cdot 10^{-4}$ already hurts accuracy
  "if λ were chosen by validation accuracy alone, does the hidden-state advantage disappear?" Figure 6.1 data. The hyperparameter-tuning privacy cost is a follow-up; the standard answer (public data, or tuning on DP-trained validation splits) is fine

*You assume $\eta < 2/(\beta+\lambda)$ without enforcing it, and rely on clipping preserving strong convexity and smoothness with the same constants — how solid are these assumptions, and what happens to the guarantee if they fail?

- Unverified assumptions. The smoothness condition $\eta<2/(\beta+\lambda)$ is assumed, not enforced — the appendix already contains a correction where an incorrect β bound invalidated results and required re-runs with data scaling. which guarantee survives if β is violated mid-training: none — the contraction constant c changes and Thm 4.5 no longer applies. lead with the appendix experiment where the condition *is* globally enforced.

400 epochs from scratch, CIFAR-10 around 41%, no convnets, and K parallel linear models where the theory covers 1-D outputs — what does this say about relevance to modern deep learning?

- Relevance to modern deep learning. CIFAR-10 at ~41% is far below what DP-SGD with a CNN achieves, and 400 epochs from scratch is unattractive. A convex 2-layer approximation cannot reach multi-layer networks, this is a first step in extending hidden-state analysis beyond logistic regression

What problem do you really solve as per-sample gradient clipping is still necessary? per-sample clipping was the *only* reason non-convex DP-SGD ever worked, so did you remove the clipping problem or just relocate it into an approximate model?"

- Song et al. / Redberg et al. argument that clipped gradients remain gradients of a convex loss with preserved strong convexity/smoothness, since that citation pair is what lets Thm 4.5 survive clipping at all.

What problem do you really solve as the downside of poisson sampling has not been established?

    - have the concrete numbers from the throughput discussion (the 11%/4%/35% costs) ready; they're the strongest empirical rebuttal.

Do the privacy guarantees you report actually hold for the main results?
This is the most dangerous question in the chapter.

- **The unenforced assumption.** Theorem 4.5 of Bok et al. requires $\eta < 2/\beta$, and $c = |1-\eta\lambda|$ needs $\eta \leq 2/(\beta+\lambda)$. main experiments state that this is assumed "without explicitly enforcing this condition." Argue that the ε values in Table 5.1 and the main figures are not covered by the theorem.
- **A rough check.** With P=64, cross-entropy ($\beta \approx P/4 \cdot \|x\|^2 + \lambda$) and unnormalized MNIST pixels ($\|x\|^2$ around 100), β is roughly 1,600, so admissible η is about 10^{-3}. Learning-rate grid goes up to 10^−0.5.
- **Data-dependent β.** DP needs β to hold for *every* possible data point, because a neighbouring dataset can substitute any x. But the Lemma uses the data-dependent $\Sigma_i(\Lambda_i)\square\square\|x\square\|^2$, and Remark 3 takes max over j in the dataset. We took an a priori bound, such as $P \cdot \|x\|^2_max/2 + \lambda$ with a fixed feature norm.
  The Hoeffding remark about typical β is frivolous, because DP is worst-case.
- The feature-normalization appendix (norm fixed at 5) is the rescue. It is the version whose guarantees are valid, and ideally that it should be the headline result. The footnote admits this

What is the technical contribution beyond combining existing results?

The components are all prior work: the convex dual (Pilanci & Ergen), the stochastic unconstrained variant (Wang et al.), clipped-gradient convexity (Song et al., Redberg et al.), and the accountant (Bok et al.). The chapter's own lemma is a short Hessian bound. Identifying that a GLM formulation makes clipped NoisyCGD analysable. This combination hadn't been done before.

In summary, we bring together indeed those four results

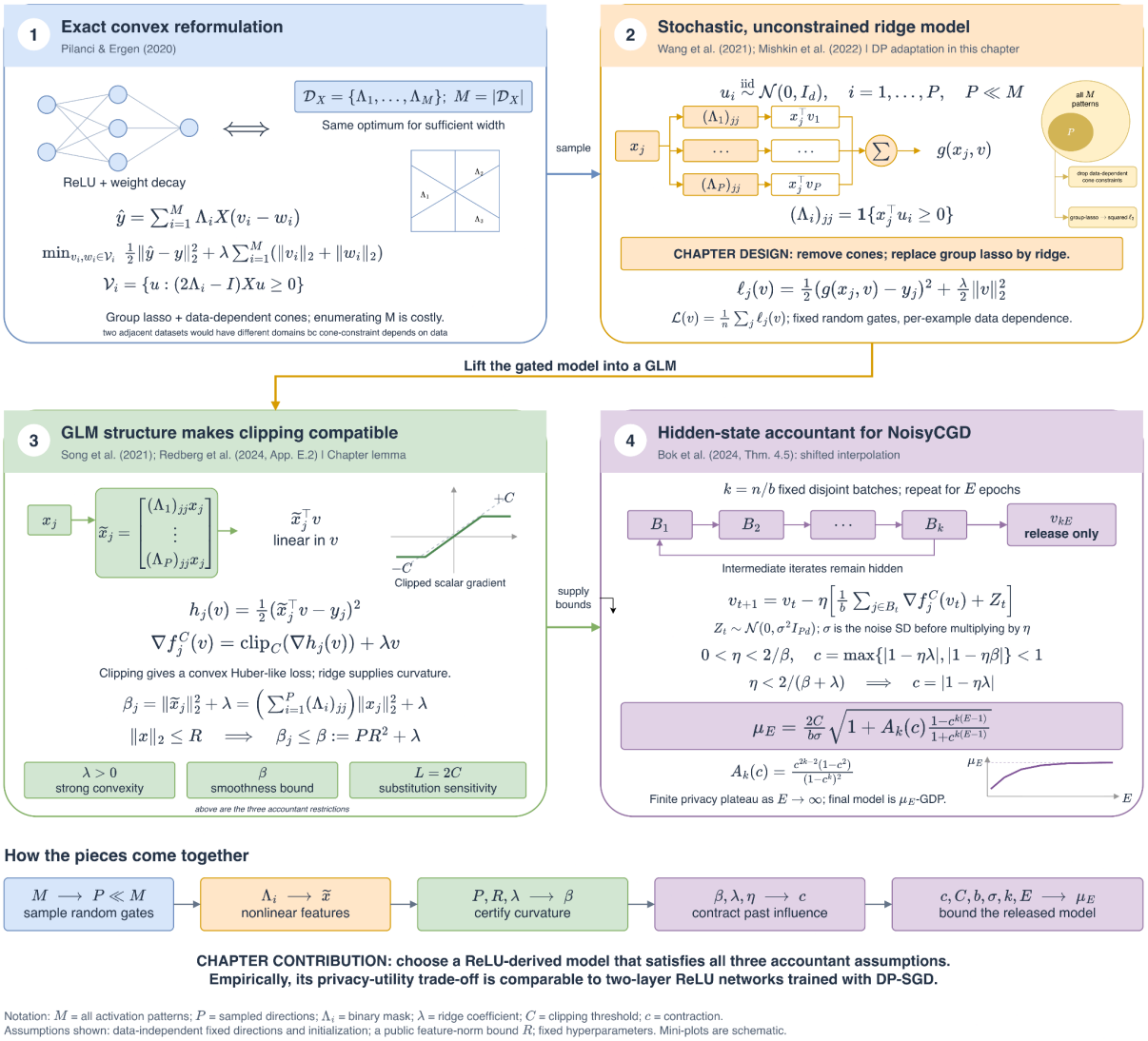
1. Convex dual (Pilanci & Ergen, 2020)
They showed that training a two-layer ReLU network with weight decay has an exact convex reformulation. It is a group-lasso problem over all ReLU activation patterns the data can produce, with cone constraints, and it attains the same optimum as the non-convex problem once the network is wide enough. The number of activation patterns grows polynomially in n only for fixed data rank, so exact enumeration is impractical for realistic datasets.
We use this as the principled starting point: our model is a relaxation of an exact reformulation of the ReLU problem, not an arbitrary convex surrogate.

2. Stochastic unconstrained variant (Wang et al., building on Pilanci & Ergen; see also Mishkin et al., 2022)
To make the convex program tractable, sample a small number P of random hyperplanes to generate a subset of activation patterns. Also, relax or drop the cone constraints, giving an unconstrained "gated ReLU"-style model that is much easier to optimize and empirically stays close to the ReLU network. This line of work was about optimization and expressivity, and never considered privacy.
We go one step further, replacing the group-lasso penalty with a squared ℓ_2 penalty and removing the data-dependent constraints. This makes the loss per-example separable and λ-strongly convex, which is what a DP analysis needs.

3. Clipped-gradient convexity (Song et al., 2021; Redberg et al., 2024)
Song et al. showed that for generalized linear models, a clipped per-example gradient is itself the gradient of another convex (Huber-like) loss, so clipping does not break convexity. Redberg et al. (their Appendix E.2) showed that strong convexity and smoothness are likewise preserved under clipping for such models.
We show that our stochastic convex model is exactly a GLM on the lifted features $\tilde{x}$□ = [(Λ₁)□□x□; …; (Λ_P)□□x□]. That lets clipping bound the gradient sensitivity while the loss keeps the convexity and smoothness constants the accountant requires, and our Lemma supplies the smoothness constant β.

4. The accountant (Bok et al., 2024)
Using shifted interpolation, they gave a tight Gaussian-DP bound on the final iterate of noisy cyclic mini-batch gradient descent (NoisyCGD). It holds for strongly convex, smooth losses with bounded gradient sensitivity, and it plateaus as the number of epochs grows instead of growing without bound. Its practical use had been limited to convex models like logistic regression, which are far less accurate than neural networks.
Because the previous three steps give a loss that is λ-strongly convex, β-smooth and 2C-sensitive, we can apply this accountant directly. That gives hidden-state guarantees for NoisyCGD with fixed disjoint batches on a model that matches two-layer ReLU networks trained with DP-SGD.

Summary: "Each piece existed separately; our contribution is designing a relaxation of the ReLU convex dual that is simultaneously tractable, a GLM, and strongly convex, so that clipping preserves the accountant's assumptions and the hidden-state analysis applies to a model with neural-network-level utility."



So what is “privacy amplification by iteration” at all?

Privacy amplification by iteration (Feldman et al., 2018) is a way to account for privacy when only the final model is released.
Each training step adds noise, and for convex, smooth losses with a small enough step size, each gradient step is a contraction: it pulls any two parameter trajectories closer together. So the influence of a data point used at some step gets progressively washed out by all the later noisy, contracting steps, and the final model reveals less about it than the full trajectory would. With strong convexity the contraction is strict (factor c < 1). As a result, the privacy bound converges to a constant as training continues, rather than growing with the number of steps as standard composition does. Bok et al.'s shifted-interpolation result is the current tight version of this analysis for NoisyCGD.

Candidate to compete with DP-SGD. DP-SGD's accounting assumes the adversary sees every intermediate model. It only gets strong guarantees through random (Poisson) subsampling, and its ε keeps growing with the number of iterations. Amplification by iteration improves on all three points:

1. Threat model: common deployment case where only the final model is published.
2. Batching: no random subsampling, so it works with fixed, disjoint, cyclic batches, which are simpler and faster to implement.
3. Training length. Its ε plateaus, so you can train longer without paying more privacy.

The catch. All of this relies on contractivity, which in turn relies on convexity. Neural network training is non-convex, so the analysis doesn't directly apply. Recent work (e.g., Annamalai, 2024) constructs non-convex losses where hidden-state amplification provably fails in the worst case. That is exactly the gap the chapter addresses: rather than extending the analysis to non-convex losses, you convexify the model so the existing analysis applies, while keeping neural-network-level utility.

One-liner for the defense: "Amplification by iteration says that noisy, contractive updates progressively forget earlier data, so the final model leaks less than the full training trajectory. It is attractive because it matches the release-only-the-final-model setting and allows efficient fixed batches, but it requires convexity, which is why we convexify the network rather than extend the analysis."

About \eta,\lambda, ηλ, and all that
* c is the contraction coefficient, c small like 0.3 is good for contraction and thus for PABI, but bad for learning. c has to be smaller than $c = \max\{|1 - \eta\lambda| , |1 - \eta\beta|\}$. As such, we say, ηλ being small is good for accuracy, but bad for privacy accounting

More questions

- **Privacy cost of tuning.** η, λ, P and the hidden width are tuned on grids, apparently by test accuracy, and that cost is not accounted for. It affects all methods equally, but so does DP-SGD's. Is the comparison fair when NoisyCGD has an extra hyperparameter (λ, via ηλ) that directly trades privacy against utility?
- **The ηλ trade-off.** $\eta\lambda = 2 \cdot 10^{-4}$ already hurts accuracy, while the privacy benefit needs ηλ to be "not too small." How narrow is the usable window, and is it dataset-dependent? This decides whether the method is usable in practice.
- **Strength of the SSP baseline.** SSP on the convex-approximation features has $P \cdot d \approx 50k$ dimensions for MNIST, and perturbing a covariance matrix of that size is hopeless. So is the 52% vs 92% gap an argument about baseline dimensionality rather than about iterative methods? Also, SSP's clipping norm is estimated non-privately from data, which you acknowledge, and CIFAR uses only 64 features.
- **Dimension and noise.** The parameter count is $d \times P \times K$, about 2M for CIFAR, while noise scales with dimension. How does the method scale to larger inputs or larger P? Appendix adds P=128 is near-optimal precisely because of this.
- **The add/remove vs substitute comparison.** The comparison to Abadi et al. via "a multiplicative difference of 2" between neighbourhood relations is loose. Substitution

corresponds to group privacy of size 2, which doesn't translate to exactly halving ε. Expect "can you make that precise?"
- **Beyond two layers.** Convex reformulations for deeper networks exist (Ergen & Pilanci), but their pattern enumeration blows up. Do you see a real path to deeper networks, or is this a dead end? Have an honest, thoughtful answer ready.

Detailed things

- **Sensitivity symbol.** Section 5.3.4 says clipping makes the loss "2L-sensitive", but the clipping constant is C. It should be 2C, or state L = C.
- **Learning-rate condition.** It is stated three ways: $\eta \in (0, 2/\beta)$ in the theorem, $\eta < 2/(\beta+\lambda)$ in the experiments, and a tuning grid on $(0, 2/(\beta+\lambda))$ in the appendix, where β already includes λ. State why $2/(\beta+\lambda)$ is needed, for $c = |1-\eta\lambda|$
- **Error-bar units.** The appendix table reports ±0.002, apparently as a fraction, while the main table reports ±0.1 in percentage points. Units differ, this is a mistake
- **Main vs normalized table.** The main table and the normalized appendix table give different numbers for the same methods (e.g., DP-SGD + Convex Approximation on MNIST at ε=1.33: 93.1 vs 92.5). Be ready to explain which is "the" result.
- **Conclusion vs chapter numbers.** The "25% utilization / 33% throughput" figures in the conclusion don't match the 11% and 35% examples in the chapter.
- **Notation.** κ (condition number) is listed in the notation summary but never used. σ also means two different things, should be cleaner
- **Figure captions.** The figures show accuracy against ε along training. Each point is a separate final model (fresh E), so each ε applies to its own model.

Chapters NoEsis and PoEtry

Chapter 6 — NoEsis (DP knowledge transfer in multi-domain LLMs)

- Deployment needs a trusted central server that sees all raw data — how does that square with the thesis's decentralization theme? And Stage-2 adapters are trained without DP: what exactly does ε=1 protect, and why is within-domain privacy out of scope?
- Removing the shared prompts in the ablation collapses Python to 4.65%, yet the main table shows only ~1-2 points of transfer gain over Share Nothing — isn't the ablation confounded by the two-stage training (which you admit)?
- Are programming languages convincing 'private domains' — who is the adversary that must not learn Python code from Java?
  Does the deduplication trick make the setting realistic or contrived?
- What do 32 DP-trained prompt tokens actually encode — did you inspect them? And why does prompt-tuning beat a rank-1 LoRA at equal parameter count, something you flag as unexplained?

- Domain transfer: GPT2 paper for between machine-translation, document-q-answering and summarization just from next-token-prediction pretraining. A second Kaplan paper “Scaling Laws for Transfer” and the “Code Llama” paper also show that pretraining on next makes coding models much better: as a quotable-chunk, pretraining from scratch only managed to obtain the loss that ntp-pretraining model got already halfway training.
  Of course, these benefits disappear at scale, and Chen et al. 2023 “Evaluating Large Language Models Trained on Code” show that with enough (programming) data, that pretraining does not help anymore.
  Conversely, pretraining with code makes models better on NLP reasoning tasks: Aryabumi et al. "To Code or Not to Code? Exploring Impact of Code in Pre-training," ICLR 2025

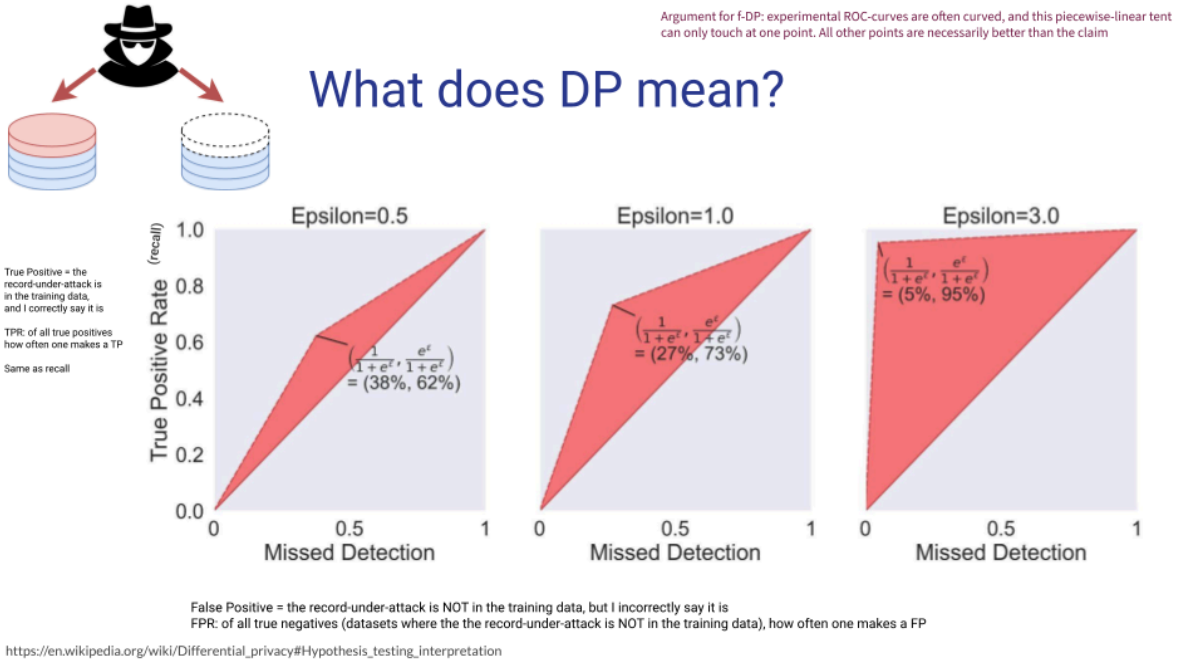
Chapter 7 — PoEtry (DP in-context learning)

- The consistency theorem holds as $J \to \infty$, but the gains are at J=4–8 — how does an asymptotic argument justify the regime you operate in? Is the bounded-interaction assumption ever validated empirically (attention weights)?
- The LLM is local and trusted, so who exactly is the adversary a DP guarantee defends against? If the querier is the data owner, what is being protected — the third parties whose emails are in the inbox?
- The ε is per token — what is the end-to-end privacy budget of a realistic multi-sentence response, and is the single-token classification setting representative of the RAG/agentic use cases you motivate?
- Is the comparison with the synthetic-data baseline fair — different privacy unit (100k training docs vs. one context example), their code re-run on the Qwen model? And how do you set the clipping bound γ, trading bias against noise?

Brief prep notes on the sharpest points:

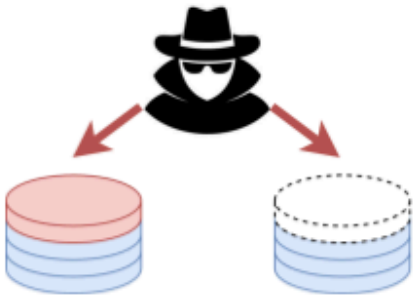
- **NoEsis, scope of the guarantee.** The two-stage design means DP protects only the shared prompts P; the domain adapters $E^{(k)}$ are trained *without* noise. Pin down exactly what ε=1 covers: cross-domain leakage via P, nothing within a domain. The defense is that the threat model is explicitly cross-domain (an adversary in domain k seeing $(P, E^{(k)})$). The trusted-server assumption is already addressed as downside in the conclusion chapter.
- **NoEsis, the ablation anomaly.** detail: "w/out common prompts" scores 4.65% on Python while Share Nothing — which also has no shared parameters — scores 68.31%. One is ablation, other is training
- **PoEtry, theory-practice mismatch.** The theorem is an asymptotic consistency result, but the selling point is the J=4–8 regime. The theorem justifies the per-example querying, the experiments justify the *regime* — and the VLM table nicely isolates the conditional-independence cost (~6–9 points) from the DP-noise cost. Note the own margin comments say the attention-analysis validation wasn't obtained.
- **PoEtry, threat model.** the inbox contains *other people's* emails, customer interactions involve *other customers* — the data owner making the query is not the only data subject. Figure 1 setting, and connect ε to what a querier can infer other

- **Both chapters share one meta-question:** "you showed MIAs in both chapters — but you also argue MIAs prove little; so what do the MIA results actually establish?" -> MIA as a sanity check complementing, never replacing, the formal guarantee



Argument for f-DP: experimental ROC-curves are often curved, and this piecewise-linear tent can only touch at one point. All other points are necessarily better than the claim

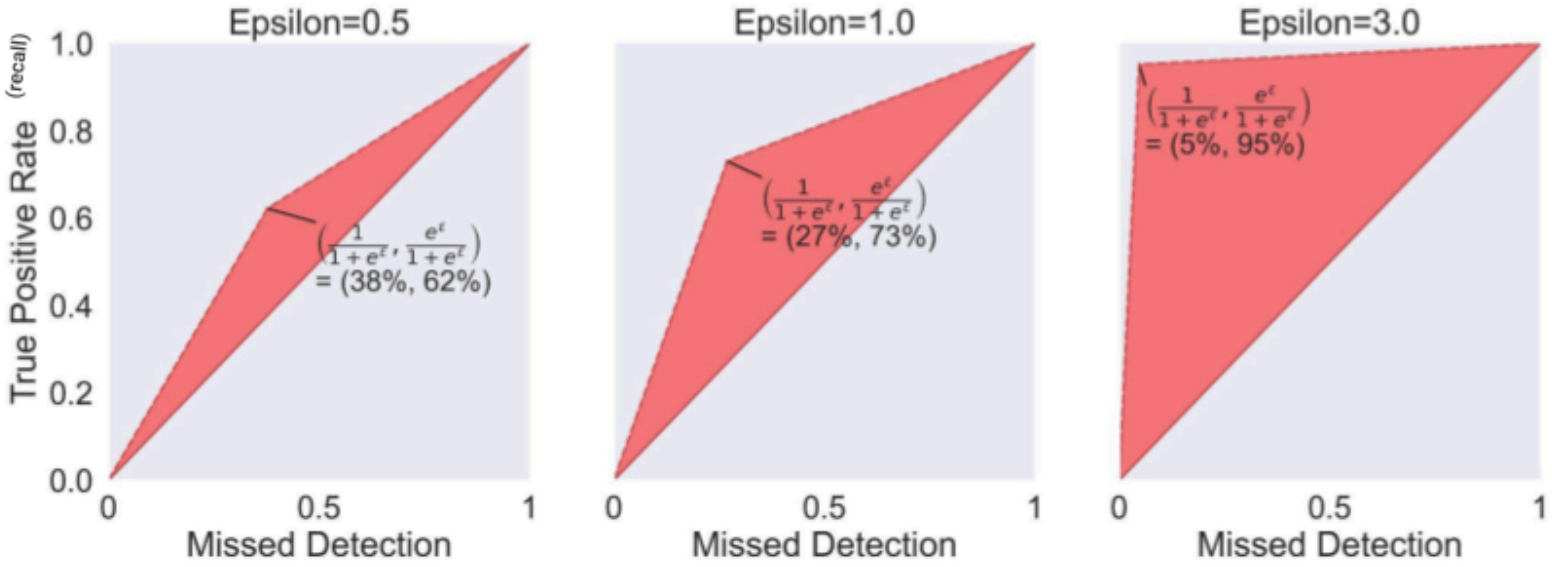
What does DP mean?



True Positive = the record-under-attack is in the training data, and I correctly say it is

TPR: of all true positives how often one makes a TP

Same as recall



False Positive = the record-under-attack is NOT in the training data, but I incorrectly say it is
FPR: of all true negatives (datasets where the the record-under-attack is NOT in the training data), how often one makes a FP

https://en.wikipedia.org/wiki/Differential_privacy#Hypothesis_testing_interpretation